

고정점 기반 프로그래밍 언어 거리: 탐색적 연구

Abstract 본 논문은 대규모 언어 모델(LLM) 기반 코드 변환의 신뢰성을 정량화하기 위해 의미 보존을 보정한 새로운 고정점 기반 프로그래밍 언어 거리 측정 프레임워크를 제안한다. 기존의 왕복 번역(RTT) 고정점 측정은 컴파일이나 실행 실패로 조기 중단된 경로가 장시간 수렴한 경로보다 오히려 가까운 거리로 오인되는 지표 역전의 한계가 있었다. 이러한 문제를 해결하기 위해 본 연구에서는 빠른 실패를 별도의 실패를 페널티로 분리하고, 생존 경로의 수렴 시간과 토큰, 구문 트리(AST) 다중뷰 구조 변화량을 결합한 SF(Stabilized and Functional) 프로파일 모델을 도입하였다. 제안한 프레임워크를 언어 왕복 번역 실험에 적용한 결과, 기존의 지표 역전을 해소하고 기능 보존과 구문 재작성의 독립성을 확인 및 모델 불변 언어 거리를 도출하였다. 결론적으로 본 연구에서 제안한 지표 체계는 고정점 거리를 확립하여, 향후 레저시 소프트웨어 현대화 및 자동 언어 이식 시스템의 안정성 검증에 크게 기여할 것으로 기대된다.

주요어: 프로그래밍 언어 거리, 왕복 번역(RTT), 확률적 고정점, 코드 대형 언어 모델, 다중뷰 지표, 추상 구문 트리(AST)

1. 서론

대규모 언어 모델(LLM)을 이용한 코드 변환은 단일 언어 합성을 넘어 다언어 간 의미론적 관계를 탐색하는 새로운 도구로 주목받고 있다. 특히 언어 A 의 코드를 B 로 변환한 뒤 다시 A 로 복원하는 왕복 번역(Round-Trip Translation, RTT)을 반복 적용할 때, 코드가 더 이상 변하지 않는 고정점(Fixed-point)에 도달하는 성질을 적용하여 두 언어 사이의 구조적, 의미적 거리를 규명하고자 한다[1].

그러나 기존의 고정점 기반 거리 측정에는 중대한 결함이 존재한다. 첫 번째 왕복에서 문법 오류나 실행 오류로 실패하여 즉시 중단된 경로는, 의미를 완벽히 유지하며 여러 차례의 세부 재작성을 거쳐 안정화된 경로보다 훨씬 작은 반복 횟수(예: 1회)를 기록한다. 이는 '더 빠르게 파괴된 변환이 더 가까운 언어 거리'로 평가되는 지표 역전 현상을 야기한다. 실제로 기존 파일럿 실험에서 특정 모델의 Java 및 Python 경로는 의미 보존율이 0%였음에도 평균 반복 횟수가 1.0에 수렴하여 C보다 왜곡되어 나타나는 현상이 관찰되었다.

더불어 단일 코드 유사도(예: MOSS)에 의존하는 측정은 표면적 어휘 일치와 실행 의미, 구문 구조 간의 불일치를 포착하지 못한다[4, 3]. 예컨대 C++에서 Python으로 왕복하는 과정에서 함수 분리와 루프 변경이 일어나 표면 유사도가 급감하더라도 프로그램의 기능적 정합성은 100% 유지될 수 있다.

본 연구는 이러한 한계를 살펴보고 개선 가능성을 모색하기 위해 다음 질문을 중심으로 접근한다:

1. 빠른 실패를 별도의 실패율로 분리하여 조기 중단이 짧은 거리로 오인되는 현상을 완화할 수 있는가?
2. 성공한 경로의 수렴 속도가 비슷하더라도, 다중뷰(토큰 집합, 순서, 구문 트리) 지표가 서로 다른 구조적 변화 정보를 보여주는가?
3. 현재의 기초 실험 체계가 언어 간 차이를 관찰하는 지표로서 어떤 가능성과 한계를 지니는가?

본 논문의 주요 내용은 다음과 같다. 첫째, 왕복 번역 과정을 확률적 전이 관점에서 모델링하고 유한 확인 왕복을 결합한 기초적인 SF(Stabilized and Functional) 판정 기준을 제안한다. 둘째, 실패율 d_{SF} , 수렴 시간 τ , 다중뷰 구조

변화량 Δ_0 를 나누어 기록하는 거리 프로파일 $D_\theta(c)$ 를 통해 조기 실패 왜곡을 줄이고자 시도한다. 셋째, 171건의 기초 실험 데이터를 바탕으로 지표의 특성을 관찰하고, 현 단계의 한계점과 향후 연구 로드맵을 정리한다.

2. 이론적 배경 및 메트릭 체계

2.1. 확률적 왕복과 의미 등가류

경로 $A \rightarrow B \rightarrow A$ 의 1회 왕복 연산자를 F_θ 라 하자. θ 는 모델, 프롬프트, 디코딩 설정 및 실행 환경을 포괄하는 조건 벡터다. 디코딩 및 하드웨어 환경에서 나타날 수 있는 확률적 요인을 ξ_t 라 할 때, 복원 코드의 전이 과정은 다음과 같이 표현할 수 있다:

$$X_{t+1} = F_\theta(X_t, \xi_t), \quad P_\theta(x, U) = \Pr\{F_\theta(x, \xi) \in U\}. \quad (1)$$

모든 유효 입력에 대해 동일하게 동작하는 코드를 의미 동치($\equiv_{\text{sem}}$)로 정의하고, 원본 C++ 코드 x_0 의 의미 등가류를 $\mathcal{E}_0 = \{x : x \equiv_{\text{sem}} x_0\}$ 라 하자. 이론적인 고정점은 $\mathcal{E}_0$ 안에서 이후 왕복에서도 의미와 정규화 표현이 안정적으로 유지되는 상태 집합 $\mathcal{A}_\theta \subset \mathcal{E}_0$ 에 도달하는 것이다.

그러나 모든 입력에 대한 완전한 동치성과 무한 왕복 안정화를 직접 판별하기는 어렵기 때문에, 본 연구에서는 주어진 테스트 픽스처(Fixture)와 유한한 확인 왕복(Confirmation cycles)을 활용한 조작적 관찰 방식을 취한다.

2.2. SF(Stabilized and Functional) 판정 기준

공백과 주석을 제거한 정규화 토큰 해시를 $h(x)$ 라 할 때, 후보 안정화 시점 τ_c 는 최초로 인접 왕복 해시가 일치하는 시점이다:

$$\tau_c = \min\{1 \leq t \leq K : h(x_t) = h(x_{t-1})\}, \quad K = 10. \quad (2)$$

후보가 나타나지 않으면 τ_c 는 정의되지 않는다. 후보 시점 이후 c 회의 추가 확인 왕복을 수행하며, V_t 를 왕복 t 까지의 모든 중간 언어 및 복원 C++ 코드가 컴파일과 단위 테스트를 통과한 사건이라 할 때, SF 성공 사건 S_c 는 다음과 같이

정의된다:

$$S_c = \{\tau_c \text{ 존재}\} \cap V_{\tau_c+c} \cap \bigcap_{i=1}^c \{h(x_{\tau_c+i}) = h(x_{\tau_c})\}. \quad (3)$$

즉, 본 연구에서 정의한 SF 성공은 (1) 상한 K 내 후보 안정화 도달, (2) 후보 도달까지 모든 변환 단계의 테스트 통과, (3) c 회의 추가 확인 왕복 동안 해시와 기능의 유지를 동시에 확인하는 기준이다.

2.3. 거리 프로파일과 다중류 구조 변화량

평가 가능한 전체 실험 단위 수를 N 이라 할 때, 실패 거리 $d_{SF}(c)$ 와 성공률 $p_{SF}(c)$ 는 다음과 같다:

$$p_{SF}(c) = \frac{\#S_c}{N}, \quad d_{SF}(c) = 1 - p_{SF}(c). \quad (4)$$

d_{SF} 는 컴파일 오류, 오답, 런타임 오류, 타임아웃뿐 아니라 확인 단계에서의 해시 변경까지 포함한다.

성공한 관측(S_c)에 한하여 수렴 시간 $\tau = \tau_c$ 와 원본 대비 구조 변화량 Δ_0 를 산출한다. 구조 변화량은 세 가지 관점의 유사도 함수 s_j 를 통해 $\Delta_0^{(j)} = 1 - s_j(x_0, x_{\tau_c})$ 로 계산한다:

1. **Token Multiset Dice** (s_{Dice}): 토큰의 다중집합 겹침 비율 $2|A \cap B|/(|A| + |B|)$ 로, 순서 변경에는 둔감하지만 어휘의 추가·삭제·치환을 반영한다.
2. **Token Sequence Ratio** (s_{seq}): difflib Sequence-Matcher(autojunk 비활성화)를 기반으로 블록 순서와 문장 재배치를 민감하게 감지한다.
3. **AST TSED** (s_{AST}): tree-sitter-cpp를 이용해 식별자를 추상화하고 연산자와 리터럴 값을 보존한 상태에서, APTED 알고리즘(삽입/삭제/변경 비용 1)으로 트리 편집 거리(TED)를 구한 뒤 $1 - TED/\max(|T_A|, |T_B|)$ 로 정규화한다[4].

최종 언어 거리 프로파일은 이들을 결합한 벡터로 나타낸다:

$$D_\theta(c) = \left(d_{SF}(c), \text{med}(\tau \mid S_c), \left(\text{med}(\Delta_0^{(j)} \mid S_c) \right)_{j \in \{Dice, seq, AST\}} \right) \quad (5)$$

본 연구는 성급하게 단일 점수로 합산하기보다는, 실패율과 성공 조건부 구조 변화량을 독립된 요소로 나누어 관찰하는 방식을 취한다.

3. 실험 설정 및 감사 체계

3.1. 코퍼스 및 실행 환경

백준 온라인 저지(BOJ) 기반의 IPOP 코퍼스 20문제에 대해 사전 실행 검증(validate-corpus --execute)을 수행하였다. 입력 첫 줄의 포맷 결함으로 reference.cpp가 실패한 IPOP_1260을 제외하고, 19개 문제가 검증을 통과하였다. 19개 문제에 대해 C, Java, Python의 3개 경로를 3회 독립 반복하여 총 $19 \times 3 \times 3 = 171$ 개의 관측 단위를 수집하였다.

추론 모델은 Qwen2.5-Coder 7B Instruct (Q4_K_M, LM Studio 0.4.24.0)를 사용하였으며, 디코딩 설정은 temperature=0, max_tokens=4096, seed=20260911로 고정하였다. 컴파일러 및 런타임은 GCC 15.2.0 (C++17, C11), OpenJDK 21.0.12.1, Python 3.14.7을 사용하였고, 컴파일 및 각 픽스처당 실행 제한 시간은 30초로 제한하였다.

3.2. 전수 감사 및 재현성 검증

본 실험의 171개 관측 단위 전체에 대해 원본 응답, 실행 파일, 테스트 결과, 해시 이력을 대조하는 완료 감사(completion_audit.json)를 수행하였다. 총 1,602회의 논리 요청과 실제 API 시도에서 네트워크 오류, 응답 절단, 출력 형식 오류는 발생하지 않아, 인프라 문제 없이 기초 실험이 완료되었음을 확인하였다.

통계적 신뢰구간은 문제 간 변동성을 고려하기 위해, 동일 문제의 3개 경로 및 3회 반복을 하나의 묶음으로 재표집하는 문제 단위 대응 부트스트랩(Pairwise cluster bootstrap, 2,000회, seed=20260911)을 적용하였다.

4. 실험 결과 및 분석

4.1. 성공률 및 실패 거리의 분리

확인 왕복 5회($c = 5$) 기준 각 경로의 성공률 p_{SF} 와 실패 거리 d_{SF} , 그리고 경로 간 대응 차이의 95% 부트스트랩 신뢰구간은 표 1과 같다.

관측된 성공률은 Python(70.2%) > Java(61.4%) > C(45.6%) 순으로 나타났으며, 실패 거리는 C(0.544)가 가장 컸다. 그러나 문제 단위 대응 부트스트랩 분석 결과, 모든 경로 쌍 간의 차이 신뢰구간이 0을 포함하였다. 특히 C와 Python 간의 차이 구간 상한은 0.0%p에 걸쳐 있어, 현재의 19개 문제 표본으로는 언어 경로 간의 우열을 단정하기 어렵다.

4.2. 실패 원인 관찰과 왜곡 완화

전체 실패 70건 중 68건(97.1%)은 기능 오류(오답 33건, 컴파일 오류 19건, 런타임 오류 13건, 시간 초과 3건)였으며, 2건은 확인 단계의 해시 변경이었다.

컴파일 오류의 한 예로 IPOP_10988의 C 경로는 세 반복 모두 stdbool.h를 포함하지 않은 채 bool과 true/false를 사용하여 C11 컴파일에 실패하였다. 기존 지표 체계였다면 1회 만에 중단된 이 실패가 '가장 가까운 거리'로 처리되었을 것이나, 제안한 프로파일에서는 이를 d_{SF} 의 실패율로 집계하고 성공 조건부 수렴 시간 τ 에서 제외함으로써 조기 실패가 짧은 거리로 오인되는 현상을 방지할 수 있었다.

4.3. 확인 왕복의 역할 ($c = 0$ 대 $c = 1 \dots 5$)

후보 안정화만 요구한 $c = 0$ 에서는 104건이 성공하였으나, 첫 번째 확인 왕복($c = 1$)에서 3건이 탈락하였다. IPOP_9935에서 C 경로는 확인 단계 오답으로, Java와 Python 경로는 테스트를 통과한 상태에서 코드 표현(입출력 설정 및 버퍼 조정 등)이 미세하게 바뀌며 해시가 변경되었다.

Table 1: 확인 5회 기준 생존 보장 프로파일의 성공률, 실패 거리 및 경로 간 차이

경유 경로	성공/평가	p_{SF} [95% CI]	d_{SF} [95% CI]	경로 간 차이 (Δp_{SF}) [95% CI]
cpp-to-c	26 / 57	45.6% [22.8%, 66.7%]	54.4% [33.3%, 77.2%]	—
cpp-to-java	35 / 57	61.4% [40.4%, 84.2%]	38.6% [15.8%, 59.6%]	C - Java: -15.8%p [-43.9%, 12.3%]
cpp-to-python	40 / 57	70.2% [50.9%, 89.5%]	29.8% [10.5%, 49.1%]	C - Python: -24.6%p [-50.9%, 0.0%]
—	—	—	—	Java - Python: -8.8%p [-31.6%, 15.8%]

$c = 1$ 이후 $c = 5$ 까지는 추가 탈락 없이 101건이 동일하게 유지되었다. 이는 단 1회의 일치만으로는 안정적인 고정점으로 확정하기 어려우며, 추가 확인 과정이 유의미한 필터로 작용함을 보여준다. 한편 $c \geq 1$ 에서 관찰된 평탄한 결과는 향후 확장 연구에서 확인 횟수를 1~2회 수준으로 줄여 실험 부담을 완화할 수 있는 가능성을 제시한다.

4.4. 성공 조건부 수렴 속도와 다중류 구조 변화

$c = 5$ 성공 관측 101건에 대해 산출한 수렴 시점 τ 및 구조 변화량 중앙값은 표 2와 같다.

세 경로 모두에서 τ 의 중앙값은 2.0으로 같았다. 즉, 왕복 번역이 성공적으로 수렴하는 경우 경로 간 도달 횟수에는 큰 차이가 나타나지 않았다.

반면 구조적 변화량 Δ_0 는 언어별로 상이한 양상을 보였다. 특히 Python 경로는 AST 변화량 중앙값이 0.482로 Java(0.212)에 비해 높게 나타났다. 예컨대 IPOP_10988 Python 경로는 $\tau = 3$ 에서 SF 조건을 통과하였으나, 원본의 단일 main 루프가 is_palindrome 함수로 분리되고 양방향 인덱스 기반 while 루프로 재작성되면서 AST 변화량이 0.9211에 달했다. 이는 프로그램의 기능이 유지되더라도 언어적 특성에 따라 코드 구조는 크게 달라질 수 있음을 보여주며, 기능 보존과 구조 보존을 별개의 지표로 살펴볼 필요가 있음을 시사한다.

5. 지표 적합성 검토 및 연구의 한계

현재 정립된 SF 지표 체계가 프로그래밍 언어 간의 거리를 온전히 설명하는 수준인지 검토한 결과는 다음과 같다.

5.1. 초기 검증에서 관찰된 유용성

1. **조기 실패 왜곡 완화:** 실패율 d_{SF} 와 조건부 통계 (τ, Δ_0)를 나누어 기록함으로써, 컴파일 오류나 오답으로 조기 중단된 경로가 짧은 거리로 오인되는 현상을 방지할 수 있었다.
2. **다각적 코드 변화 관찰:** 토큰 집합(Dice), 토큰 순서(Sequence), 구문 트리(AST TSED)를 함께 활용하여 어휘, 배치, 구조적 수준의 변화를 나누어 살펴볼 수 있음을 확인하였다.
3. **기초 실험의 일관성:** 전수 감사 스크립트와 실행 환경 고정을 통해 초기 실험 절차의 재현 가능성을 확보하였다.

5.2. 탐색적 연구로서의 한계점

1. **표본 크기의 제약:** 19개 문제 규모로는 부트스트랩 신뢰구간이 모두 0을 포함하여, 언어 경로 간 거리 순위를

명확하게 결론짓기 어렵다.

2. **성공 집합의 비대칭성(생존자 편향):** 표 2의 구조 변화량은 성공한 관측에서만 계산된다. C 경로는 26개, Python 경로는 40개의 서로 다른 문제 집합에서 산출되었으므로, 문제 난이도 차이에 따른 영향이 섞여 있을 수 있다.
3. **테스트 케이스의 다양성 부족:** 19개 문제 중 8개 문제는 테스트 입력이 1종류뿐이며 그 입력이 프롬프트에 예시로 제공되었다. 따라서 이들 문제에 대한 기능 보존은 일반적인 의미 동치라기보다 예시 재현에 가까운 한계가 있다.
4. **단일 모델 의존성:** Qwen2.5-Coder 7B 단일 모델로만 실험하여, C 언어의 헤더 처리 미흡과 같은 특정 모델의 생성 특성이 언어 간 거리로 혼재되어 나타날 수 있다.
5. **단일 거리 수치의 부재:** 여러 지표를 하나의 대표 거리 점수로 통합하는 기준이 아직 정립되지 않아, 계통도나 클러스터링을 즉시 도출하기는 어렵다.

따라서 현재의 지표 체계는 완성된 보편적 척도라기보다, 고정점 방식의 타당성을 점검하고 초기 실패 왜곡을 줄이기 위한 탐색적 프레임워크로 이해하는 것이 타당하다.

6. 향후 연구 방향

본 연구의 한계를 보완하고 보다 신뢰할 수 있는 언어 거리를 도출하기 위해 다음 연구를 추진하고자 한다.

6.1. 지표 모델의 정교화

1. **단일 기대 거리 모델링 검토:** 실패 상태를 흡수 상태로 설정하고 적절한 페널티를 부여하여, 의미 보존과 도달 시간을 종합적으로 고려한 기대 도달 시간 지표의 수식화를 모색한다.
2. **공동 성공 문제 비교:** 세 경로가 모두 성공한 문제 집합만을 별도로 추출하여 생존자 편향을 줄인 구조 변화량 비교를 병행한다.

6.2. 데이터셋 및 모델의 확장

1. **은닉 테스트 케이스 확충:** MultiPL-E[2] 등 공개되지 않은 테스트 케이스가 풍부한 벤치마크를 활용하고, 문제 수를 50~100개 이상으로 늘려 통계적 신뢰도를 높인다.

Table 2: SF 성공 집합에서의 수렴 시점 및 다중분류 구조 변화량 중앙값 [IQR]

경유 언어 (n)	τ 중앙값	Δ_0 Dice	Δ_0 Sequence	Δ_0 AST TSED	AST 유효율
cpp-to-c (26)	2.0 [2.0, 3.0]	0.156 [0.128, 0.254]	0.202 [0.148, 0.305]	0.360 [0.268, 0.502]	100.0%
cpp-to-java (35)	2.0 [2.0, 2.0]	0.140 [0.113, 0.148]	0.155 [0.127, 0.170]	0.212 [0.209, 0.303]	100.0%
cpp-to-python (40)	2.0 [2.0, 3.0]	0.166 [0.108, 0.208]	0.292 [0.139, 0.469]	0.482 [0.267, 0.794]	100.0%

2. **다양한 모델 교차 검증:** GPT-4o, Claude 3.5 Sonnet, CodeLlama 등 여러 계열의 모델을 적용하고, 선형 혼합효과 모형(Mixed-effects Model)을 통해 언어 고유의 효과와 모델 고유의 편향을 통계적으로 분리하는 분석을 시도한다.

[5] Rob van der Goot, Esther Ploeger, Verena Blaschke, and Tanja Samardžić. DistaLs: A comprehensive collection of language distance measures. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 307–318, 2025.

6.3. 다언어 계통도 구축 탐색

C++, C, Java, Python 외에 Rust, Go, TypeScript 등 다양한 언어로 실험 범위를 넓혀 언어 간 거리 행렬을 구성하고, 자연어 언어 거리 컬렉션[5]과 같이 프로그래밍 언어 간의 유사도 지형을 탐색하는 연구로 발전시키고자 한다.

7. 결론

본 연구는 왕복 번역 고정점을 기반으로 언어 거리를 측정할 때 나타나는 조기 실패 왜곡을 줄이고자, 실패율과 성공 조건부 구조 변화량을 분리하여 살펴보는 탐색적 연구를 수행하였다. 171건의 기초 실험을 통해 제안한 방식이 조기 실패를 걸러내고 기능 보존과 구문 재작성의 차이를 관찰하는 데 유용함을 확인하였다. 비록 작은 표본과 단일 모델을 사용한 초기 단계의 한계가 존재하지만, 본 연구는 고정점 기반 언어 거리의 기초적 틀을 마련하고 향후 다모델·대규모 연구로 나아가는 디딤돌이 될 것으로 기대한다.

참고문헌

- [1] Miltiadis Allamanis, Sheena Panthaplackel, and Pengcheng Yin. Unsupervised evaluation of code LLMs with round-trip correctness. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 1050–1066, 2024.
- [2] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3654–3673, 2023.
- [3] Shuo Ren, Daya Rao, Lin Ding, Peng Dai, Hailong Liu, Chao Lu, and Xiaodong Chen. CodeBLEU: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*, 2020.
- [4] Yewei Song, Cedric Lothritz, David Tang, Tegawendé Bisseyandé, and Jacques Klein. Revisiting code similarity evaluation with abstract syntax tree edit distance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 38–46, 2024.